INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

Banco de Dados II

CAROLINA FRANCISCA CLARO OLIVEIRA

POSTGRESQL: CARACTERÍSTICAS, APLICAÇÕES E EXEMPLOS DE UTILIZAÇÃO

Campos do Jordão - SP
2026

Sumário

1. Introdução 3
2. Onde o PostgreSQL é utilizado 3
3. Principais características 4
4. Requisitos e instalação 6
5. Exemplos de consultas SQL 6
6. Conclusão 9
Referências bibliográficas 11

1. Introdução

O PostgreSQL é um Sistema Gerenciador de Banco de Dados (SGBD) objeto-relacional, gratuito e de código aberto. Ele utiliza e estende a linguagem SQL e oferece mecanismos destinados ao armazenamento, à organização, à consulta e à proteção de dados. O projeto tem origem no POSTGRES, desenvolvido a partir de 1986 na Universidade da Califórnia, em Berkeley, e evoluiu continuamente até se tornar uma das soluções de banco de dados de código aberto mais consolidadas da atualidade.

Um SGBD tem a função de intermediar o acesso entre aplicações e os dados armazenados. Em vez de cada programa controlar diretamente arquivos e registros, o banco de dados fornece recursos padronizados para criação de tabelas, inserção e alteração de informações, consultas, controle de acesso, integridade e recuperação. No PostgreSQL, essas operações podem ser executadas por SQL e por ferramentas gráficas, como o pgAdmin, ou por aplicações desenvolvidas em linguagens como Java, Python, C#, JavaScript e outras.

Entre os pontos que diferenciam o PostgreSQL estão a confiabilidade, a integridade transacional, o suporte a consultas complexas, a extensibilidade e o controle de concorrência por múltiplas versões (MVCC). A documentação oficial também destaca recursos como chaves estrangeiras, gatilhos (triggers), views, tipos de dados avançados, índices, replicação e mecanismos de segurança.

Na data de elaboração deste trabalho, a versão estável principal do PostgreSQL é a 18, enquanto a versão 19 ainda se encontra em ciclo beta. Esse detalhe demonstra que o projeto mantém desenvolvimento ativo e lançamentos regulares, embora versões beta não sejam recomendadas para ambientes de produção.

2. Onde o PostgreSQL é utilizado

Por ser um banco de dados de propósito geral, o PostgreSQL pode ser empregado em projetos de diferentes portes. Ele aparece tanto em aplicações acadêmicas e sistemas internos quanto em serviços web, sistemas corporativos e ambientes que processam grandes volumes de dados. Sua licença permissiva permite uso privado, acadêmico e comercial.

2.1 Aplicações web e APIs

Sistemas web precisam armazenar informações como usuários, produtos, pedidos, pagamentos, permissões e históricos de operação. O PostgreSQL é frequentemente escolhido como banco de dados de aplicações web e APIs porque oferece transações, restrições de integridade, índices e consultas relacionais, além de suporte a estruturas JSON e JSONB para dados semiestruturados.

2.2 Sistemas empresariais

Sistemas de gestão, como ERP, CRM, controle de estoque, folha de pagamento e plataformas de atendimento, podem utilizar PostgreSQL para concentrar dados de diversas áreas. Recursos como chaves primárias, chaves estrangeiras, constraints e transações contribuem para manter consistência mesmo quando diferentes usuários e aplicações acessam o banco simultaneamente.

2.3 Análise de dados e relatórios

O PostgreSQL oferece funções de agregação, agrupamento, ordenação, subconsultas, views e funções de janela, o que permite construir relatórios e apoiar análises de dados. Em cenários de Business Intelligence, ele também pode atuar como fonte de dados para ferramentas de visualização e dashboards.

2.4 Dados geográficos

Com a extensão PostGIS, o PostgreSQL pode armazenar, indexar, consultar e processar informações geoespaciais. Dessa forma, pode ser usado em mapas, logística, planejamento urbano, agricultura, localização de serviços e outras aplicações baseadas em coordenadas e geometria.

2.5 Computação em nuvem

O PostgreSQL também pode ser executado em servidores locais, máquinas virtuais, contêineres e serviços gerenciados em nuvem. Essa flexibilidade permite que a mesma tecnologia seja utilizada desde ambientes de desenvolvimento até arquiteturas de produção com alta disponibilidade e replicação.

3. Principais características

As características do PostgreSQL abrangem aspectos de modelagem, segurança, desempenho e extensibilidade. A seguir são apresentadas algumas das mais importantes.

Característica	Descrição

Código aberto e licença permissiva	Pode ser utilizado, modificado e distribuído gratuitamente para fins privados, comerciais e acadêmicos.
SQL e consultas complexas	Suporta grande parte do padrão SQL e recursos avançados para consulta e manipulação de dados.
Integridade de dados	Oferece PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK e outras restrições.
Transações ACID	Permite que operações relacionadas sejam confirmadas ou desfeitas de forma consistente.
MVCC	Mantém versões de dados para melhorar a concorrência entre leituras e escritas.
Índices	Inclui B-tree, Hash, GiST, SP-GiST, GIN e BRIN, além de índices parciais e por expressão.
Tipos avançados	Suporta arrays, UUID, JSON/JSONB, XML, intervalos, tipos geométricos e tipos definidos pelo usuário.
Extensibilidade	Permite criar funções, tipos, operadores, linguagens procedurais e instalar extensões.
Replicação e recuperação	Oferece replicação física e lógica, WAL e recuperação para pontos específicos no tempo.
Segurança	Possui autenticação, permissões por objeto, segurança em nível de linha e diferentes métodos de autenticação.

O controle de concorrência merece destaque. O PostgreSQL utiliza MVCC (Multi-Version Concurrency Control), mecanismo em que cada instrução trabalha com uma visão consistente dos dados. Esse modelo reduz conflitos entre operações de leitura e escrita e favorece o funcionamento de ambientes com vários usuários simultâneos.

Outro ponto importante é a extensibilidade. O banco permite adicionar tipos de dados, funções, operadores, métodos de índice e extensões. O próprio PostGIS é um exemplo conhecido: ele acrescenta recursos geográficos ao PostgreSQL sem transformar o banco em um produto separado.

4. Requisitos e instalação

O PostgreSQL possui pacotes e instaladores para os principais sistemas operacionais. A página oficial disponibiliza opções para Linux, macOS, Windows, BSD e Solaris, além do código-fonte para compilação manual [4]. A documentação de plataformas suportadas cita arquiteturas como x86, PowerPC, ARM, RISC-V e outras, dependendo do sistema operacional e da versão utilizada.

Não existe um único requisito de hardware que seja adequado para todos os ambientes. A quantidade necessária de memória RAM, espaço em disco e capacidade de processamento varia de acordo com o volume de dados, quantidade de conexões, complexidade das consultas, número de usuários e necessidade de replicação ou backup. Para uso acadêmico e desenvolvimento, uma instalação local comum costuma ser suficiente; já ambientes de produção devem ser dimensionados conforme a carga real.

4.1 Requisitos básicos para ambiente de estudo

- Sistema operacional compatível, como Windows, Linux ou macOS.
- Espaço em disco para o servidor, ferramentas e bases de dados que serão criadas.
- Memória RAM e processador compatíveis com a carga de trabalho pretendida.
- Permissão de administrador para instalação e configuração do serviço, quando necessária.
- Uma ferramenta cliente, como psql ou pgAdmin, para executar comandos e administrar o banco.

No Windows, o instalador disponibilizado pela comunidade por meio da EDB inclui o servidor PostgreSQL, o pgAdmin e o StackBuilder. O pgAdmin fornece uma interface gráfica para criação e administração de bancos, tabelas, usuários e consultas.

5. Exemplos de consultas SQL

Para demonstrar a utilização do PostgreSQL, será considerada uma pequena base de dados de uma loja. O exemplo contém clientes e pedidos e permite utilizar operações de definição, inclusão, consulta, alteração, exclusão e junção de dados.

5.1 Criação das tabelas

```
CREATE TABLE clientes (
    id_cliente INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE,
    cidade VARCHAR(80)
);
```

```
CREATE TABLE pedidos (
   id_pedido INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   id_cliente INTEGER NOT NULL,
   data_pedido DATE NOT NULL,
   valor_total NUMERIC(10,2) CHECK (valor_total >= 0),
   CONSTRAINT fk_cliente
      FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);
```

Nesse exemplo, as tabelas utilizam chave primária, chave estrangeira, restrição de unicidade, obrigatoriedade e validação de valor. As constraints são importantes porque impedem que dados incompatíveis com as regras definidas sejam gravados [9].

5.2 Inserção de registros

```
INSERT INTO clientes (nome, email, cidade) VALUES
('Ana Souza', 'ana@email.com', 'Campos do Jordão'),
('Bruno Lima', 'bruno@email.com', 'Taubaté'),
('Carla Mendes', 'carla@email.com', 'São Bento do Sapucaí');

INSERT INTO pedidos (id_cliente, data_pedido, valor_total) VALUES
(1, '2026-09-10', 250.00),
(1, '2026-09-18', 180.50),
(2, '2026-09-20', 420.00);
```

5.3 Consulta simples

```
SELECT *
FROM clientes;
```

O comando SELECT recupera dados armazenados. O asterisco indica que todas as colunas da tabela devem ser retornadas.

5.4 Consulta com filtro e ordenação

```
SELECT nome, cidade
FROM clientes
WHERE cidade = 'Campos do Jordão'
ORDER BY nome;
```

A cláusula WHERE restringe os registros retornados e ORDER BY define a ordenação do resultado.

5.5 Atualização de dados

```
UPDATE clientes
SET cidade = 'Pindamonhangaba'
WHERE id_cliente = 2;
```

O UPDATE altera dados existentes. É importante utilizar uma condição adequada no WHERE para evitar a atualização involuntária de todas as linhas da tabela.

5.6 Exclusão de dados

```
DELETE FROM pedidos
WHERE id_pedido = 3;
```

O DELETE remove registros. Assim como no UPDATE, a cláusula WHERE define quais linhas serão afetadas.

5.7 Junção entre tabelas

```
SELECT
   c.nome,
   p.data_pedido,
   p.valor_total
FROM clientes c
INNER JOIN pedidos p
   ON c.id_cliente = p.id_cliente
ORDER BY p.data_pedido;
```

O INNER JOIN relaciona registros das tabelas clientes e pedidos por meio do campo id_cliente. Esse tipo de consulta demonstra uma das principais vantagens do modelo relacional: manter dados organizados em tabelas distintas e combiná-los quando necessário.

5.8 Agrupamento e função de agregação

```
SELECT
   c.nome,
   COUNT(p.id_pedido) AS quantidade_pedidos,
```

```
    SUM(p.valor_total) AS total_comprado
FROM clientes c
LEFT JOIN pedidos p
    ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nome
ORDER BY total_comprado DESC NULLS LAST;
```

Nesse exemplo, COUNT contabiliza a quantidade de pedidos e SUM calcula o total comprado. O GROUP BY agrupa os registros por cliente. O LEFT JOIN permite que clientes sem pedidos também apareçam no resultado.

5.9 Exemplo de transação

```
BEGIN;

UPDATE pedidos
SET valor_total = valor_total * 0.90
WHERE id_pedido = 1;

COMMIT;
```

Uma transação reúne uma ou mais operações como uma unidade lógica. Caso seja necessário desfazer alterações antes da confirmação, pode-se utilizar ROLLBACK no lugar de COMMIT. O suporte transacional é essencial para preservar consistência em operações que não podem ficar parcialmente concluídas.

6. Conclusão

O PostgreSQL é um sistema gerenciador de banco de dados robusto, de código aberto e adequado a diferentes tipos de aplicações. Ao longo do trabalho foram apresentadas sua origem, áreas de utilização, características, requisitos e exemplos de comandos SQL. A análise demonstra que o PostgreSQL reúne os recursos fundamentais de um banco relacional, como tabelas, relacionamentos, restrições, transações e consultas, juntamente com funcionalidades avançadas, como JSON/JSONB, extensões, replicação, múltiplos tipos de índice e suporte geoespacial por meio do PostGIS.

Sua utilização em sistemas web, aplicações empresariais, análise de dados e soluções geográficas mostra que a ferramenta pode atender tanto a projetos acadêmicos quanto a sistemas profissionais de maior complexidade. Além disso, a disponibilidade para diferentes sistemas operacionais e a existência de ferramentas como psql e pgAdmin facilitam sua adoção em ambientes de desenvolvimento e estudo.

Para estudantes de Análise e Desenvolvimento de Sistemas, conhecer PostgreSQL contribui para compreender na prática conceitos como modelagem relacional, integridade referencial, consultas SQL, transações, segurança e desempenho. Dessa forma, o estudo dessa tecnologia não se limita ao aprendizado de comandos, mas também auxilia na compreensão de princípios importantes para o projeto e a manutenção de sistemas de informação.

Referências bibliográficas

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL: About**. Disponível em: https://www.postgresql.org/about/. Acesso em: 20 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation:** What Is PostgreSQL? Disponível em: https://www.postgresql.org/docs/current/intro-whatis.html. Acesso em: 20 set. 2026.

POSTGIS PROJECT. **PostGIS Manual:** Introduction. Disponível em: https://postgis.net/docs/. Acesso em: 24 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation:** Supported Platforms. Disponível em: https://www.postgresql.org/docs/17/supported-platforms.html. Acesso em: 24 set. 2026.

MICROSOFT AZURE. **O que é PostgreSQL?** Disponível em: https://azure.microsoft.com/pt-br/resources/cloud-computing-dictionary/what-is-postgresql. Acesso em: 24 set. 2026.

GOOGLE CLOUD. **What is PostgreSQL?** Disponível em: https://cloud.google.com/discover/what-is-postgresql. Acesso em: 24 set. 2026.